

Placement Portal Application

App Dev 2 – Project Report

1. Student Details

Name: Kumar Gaurav

Roll Number: 22f3003274

Email: 22f3003274@ds.study.iitm.ac.in

About Me: I am a final year B.Tech student in Electrical and Electronics Engineering at IIIT Bhubaneswar and also pursuing a B.Sc. in Data Science and Applications. I enjoy learning new technologies and solving real world problems.

2. Project Details

Project Title: Placement Portal Application-v2

Problem Statement:

To design and build a unified web-based application that streamlines the campus recruitment process. It allows students to seamlessly apply for jobs, companies to manage job postings and candidate shortlists, and administrators to oversee the entire placement process with real-time analytics.

Approach:

The app was built using a decoupled architecture with Flask as the REST API backend and Vue.js as the frontend. It allows users to log in based on their roles (Admin, Student, Company), tracks application statuses, visualizes placement statistics through interactive charts, and uses Celery/Redis for asynchronous background tasks like email reminders and CSV data exports.

3. Technologies and Frameworks Used

Flask – Core backend web framework

SQLAlchemy – Object Relational Mapper for SQLite database

SQLite – Lightweight local database for storing application data

Vue.js – Frontend reactive framework for building the Single Page Application (SPA)

Bootstrap 5 – Responsive frontend styling

Chart.js – Data visualization in dashboards

Flask-JWT-Extended – Secure token-based user authentication and role management

Celery – Handling asynchronous background tasks and scheduled cron jobs

Werkzeug – Password hashing and authentication

Redis – Message broker for Celery tasks

4. Database Schema

Tables:

User: id, email, password, name, phone, role, is_blacklisted, created_at

StudentProfile: id, user_id, roll_number, branch, year, cgpa, skills, resume, updated_at

CompanyProfile: id, user_id, company_name, location, website, hr_email, hr_phone, approval_status, created_at

PlacementDrive: id, company_id, job_title, job_description, eligibility_criteria, salary_lpa, application_deadline, drive_date, location, job_type, status, created_at, updated_at

Application: id, drive_id, student_id, status, applied_at, interview_date, feedback, history, updated_at

Placement: id, application_id, company_id, student_id, drive_id, job_title, salary_lpa, joining_date, placed_at, offer_letter

Relationships:

One-to-One: User to StudentProfile

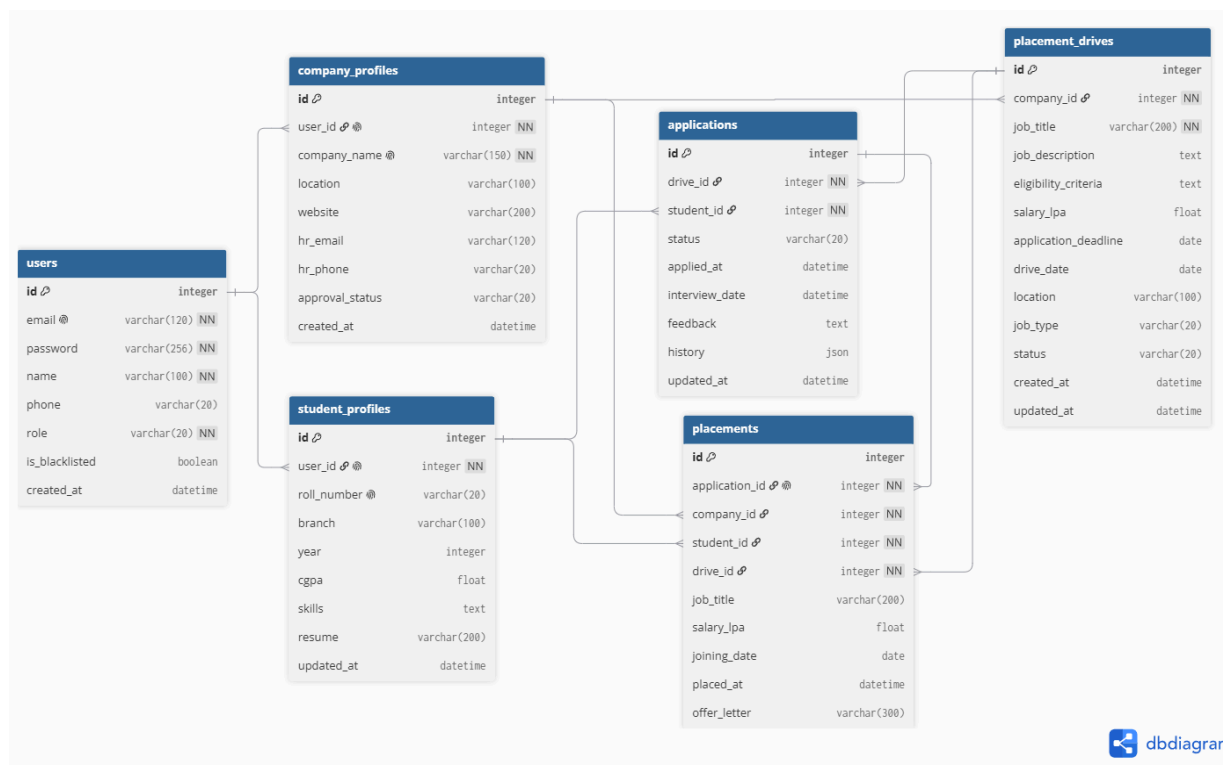
One-to-One: User to CompanyProfile

One-to-Many: CompanyProfile to PlacementDrive

One-to-Many: StudentProfile to Application

One-to-Many: PlacementDrive to Application

ER Diagram:



5. Architecture and Features

Project Structure:

/backend/app.py - main Flask application entry point

/backend/apps/models.py - database models using SQLAlchemy

/backend/apps/routes/ - Flask Blueprints for auth, student, company, and admin routes

/backend/apps/tasks.py - Celery background tasks and periodic jobs

/frontend/index.html - main SPA entry point

/frontend/components/ - Vue.js components for UI views and dashboards

Implemented Features:

- Role-based registration and secure JWT login
 - Placement drive creation and management
 - Student applications for placement drives
 - Admin approval/rejection of companies and drives
 - Student blacklisting system
 - Interactive visualization dashboard for Admins using Chart.js
 - Search functionality for students and companies
 - Role-based access control
 - Responsive UI using Bootstrap
 - Asynchronous email reminders for upcoming interviews
 - Monthly report generation
-

6. API Resource Endpoints

Some of the important endpoints are:

Endpoint	Method	Description
/api/auth/login	POST	Authenticates user and generates JWT session token
/api/auth/register	POST	Registers a new Student or Company account
/api/student/dashboard	GET	Retrieves available drives and student's application history
/api/student/profile/update	PATCH	Updates student academic profile and uploads resume
/api/company/drive/create	POST	Creates a new placement drive listing
/api/company/application/<id> /status	POST	Updates the hiring status of a student's application
/api/company/export-history	POST	Triggers a Celery task to generate a CSV of applications
/api/admin/chart-stats	GET	Fetches aggregated statistics for the Admin dashboard charts
/api/admin/companies/<id>/status	POST	Approves or rejects a company registration

7. AI / LLM Declaration

GitHub Copilot was used to create charts using ChartJs. The AI usage is around 10-15% of the development process limited to HTML suggestions and syntax correction. All core logic, integration, and final implementation were completed independently.

8. Video Demonstration

Video Link: 📺 Placement Portal Application v2- video demonstration

<https://drive.google.com/file/d/1ryPRY7kBVDewuiX1C95IeN4IE2AWodF/view?usp=sharing>